Blog

For our Blog website, we want a database table called Post with three fields: Title, Author, and Body.

Post Database Table

Post

TITLE	AUTHOR	BODY
Hello, World!	wsv	My first blog post. Woohoo!
Goals Today	wsv	Learn Django and build a blog application.
3rd Post	wsv	This is my 3rd entry.

Remember that the **models.py** file is the single, definitive source of **information about our data**, containing the **necessary fields and behaviors of the stored data**.

We can write Python in a models.py file, and the **Django ORM will translate it into SQL**.

Code

```
# blog/models.py
from django.db import models


class Post(models.Model):
    title = models.CharField(max_length=200)
    author = models.CharField(max_length=200)
    body = models.TextField()

    def __str__(self):
        return self.title
```

At the top of the file, we import the **models module** and then **create a class, Post**, that **extends it**. The Post class has **three fields** (think of them as columns) for title, author, and body. Each field must have an appropriate field type. The first two use CharField, meaning a Character Field with a maximum character length of 200, while the third uses TextField, intended for a large amount of text.

Now that our new database model exists, we need to create a new migration file and migrate the change to apply it to our database. Stop the server with `Control+c`. You can complete this two-step process with the commands below:

Shell

```
(.venv) $ python manage.py makemigrations blog
Migrations for 'blog':
  blog/migrations/0001_initial.py
    - Create model Post
(.venv) $ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, sessions
Running migrations:
  Applying blog.0001_initial... OK
```

The database is now configured, and a new `migrations` directory containing our changes exists within the `blog` app directory.

Primary keys are a standard part of relational database design. As a result, Django automatically adds an auto-incrementing primary key[101] to our database models. Its value starts at `1` and increases sequentially to `2`, `3`, and so on. The naming convention is `<table_id>`, meaning that for a Post model, the primary key column is named `post_id`.

Post Schema

```
Post
--------
post_id
title
author
body
```

As a result, under the hood, our existing Post database table has four columns/fields.

Post Database Table

```
Post
--------
POST_ID   TITLE          AUTHOR    BODY
1         Hello, World!  wsv       My first blog post. Woohoo!
2         Goals Today    wsv       Learn Django and build a blog application.
3         3rd Post       wsv       This is my 3rd entry.
```

Authentication is a common–and challenging to implement well–feature on websites.

Django has an entire built-in authentication system that we can use.

We will use the Django **auth user model.**

Post and User Schema

Post	User
post_id	user_id
title	username
author	first_name
body	last_name
	email
	password
	groups
	user_permissions
	is_staff
	is_active
	is_superuser
	last_login
	date_joined

Code

```
# blog/models.py
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(
        "auth.User",
        on_delete=models.CASCADE,
    )  # new
    body = models.TextField()

    def __str__(self):
        return self.title
```

We want the **author** field in **Post** to link to the **User** model so that **each post has an author corresponding to a user.**

We can do this by linking the User model primary key, **user_id**, to the **Post.author** field. A link like this is known as a **foreign key relationship**. Foreign keys in one table always correspond to the primary keys of a different table.

The ForeignKey field defaults to a **many-to-one relationship**, meaning one user can be the author of many different blog posts but not the other way around.

Since we have updated our database models again, we should create a new migrations file and then migrate the database to apply it.

Shell

```
(.venv) $ python manage.py makemigrations blog
Migrations for 'blog':
  blog/migrations/0002_alter_post_author.py
    - Alter field author on post
(.venv) $ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, sessions
Running migrations:
  Applying blog.0002_alter_post_author... OK
```

A second migrations file will now appear in the `blog/migrations` directory that documents this change.

Admin

We need a way to access our data: enter the Django admin! First, create a superuser account by typing the command below and following the prompts to set up an email and password. Note that typing your password will not appear on the screen for security reasons.

Shell

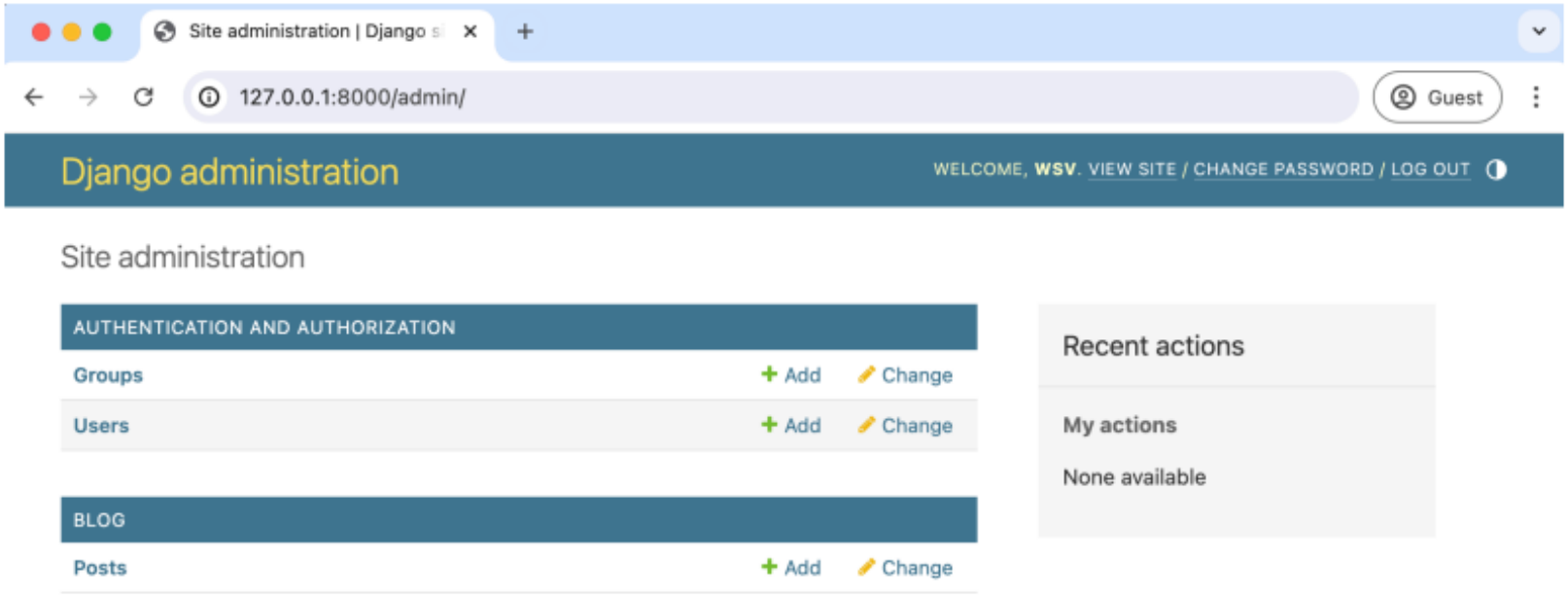
```
(.venv) $ python manage.py createsuperuser
Username (leave blank to use 'wsv'): wsv
Email:
Password:
Password (again):
Superuser created successfully.
```

Code

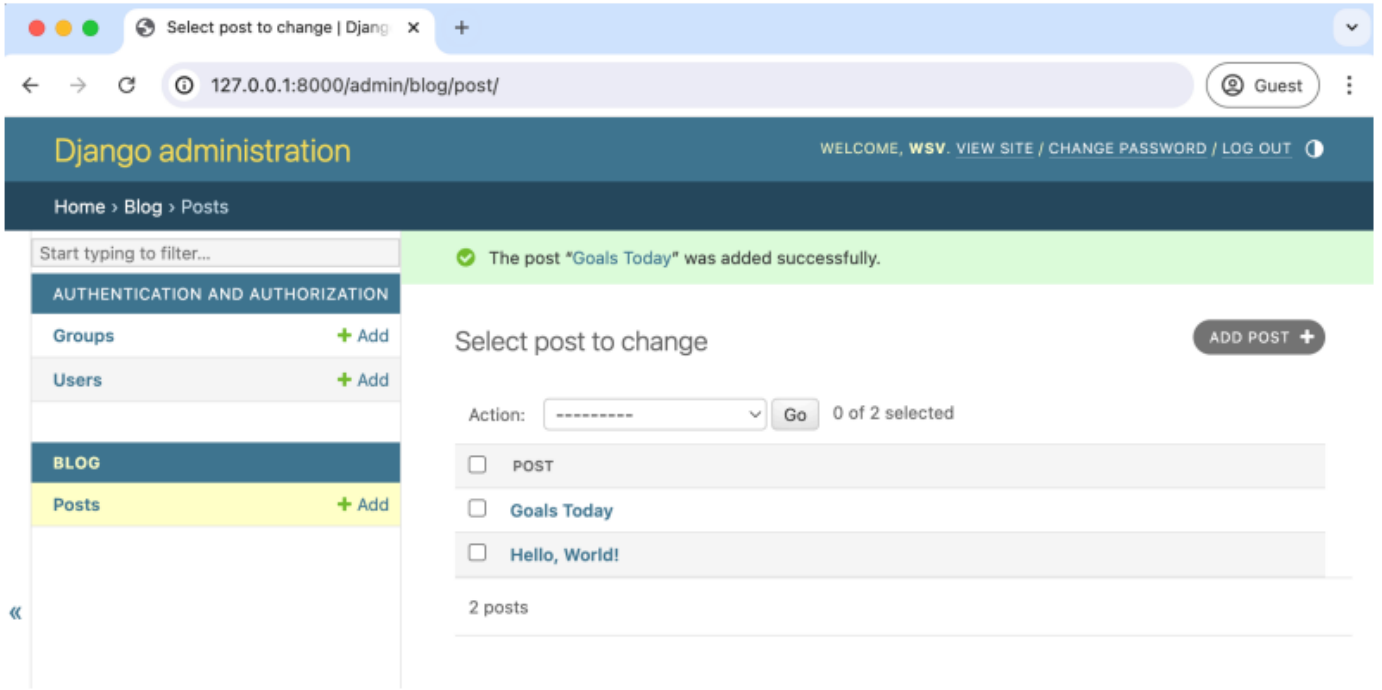
```
# blog/admin.py
from django.contrib import admin
from .models import Post

admin.site.register(Post)
```

If you refresh the page, you'll see the update.



Admin Homepage



Admin Change List with Two Posts

Although our model has three fields, the Django admin defaults to displaying whatever is in the `__str__` method in the list view. In our case, that is the `title` field. However, it is quite straightforward to customize the admin further to display all three fields.

Code

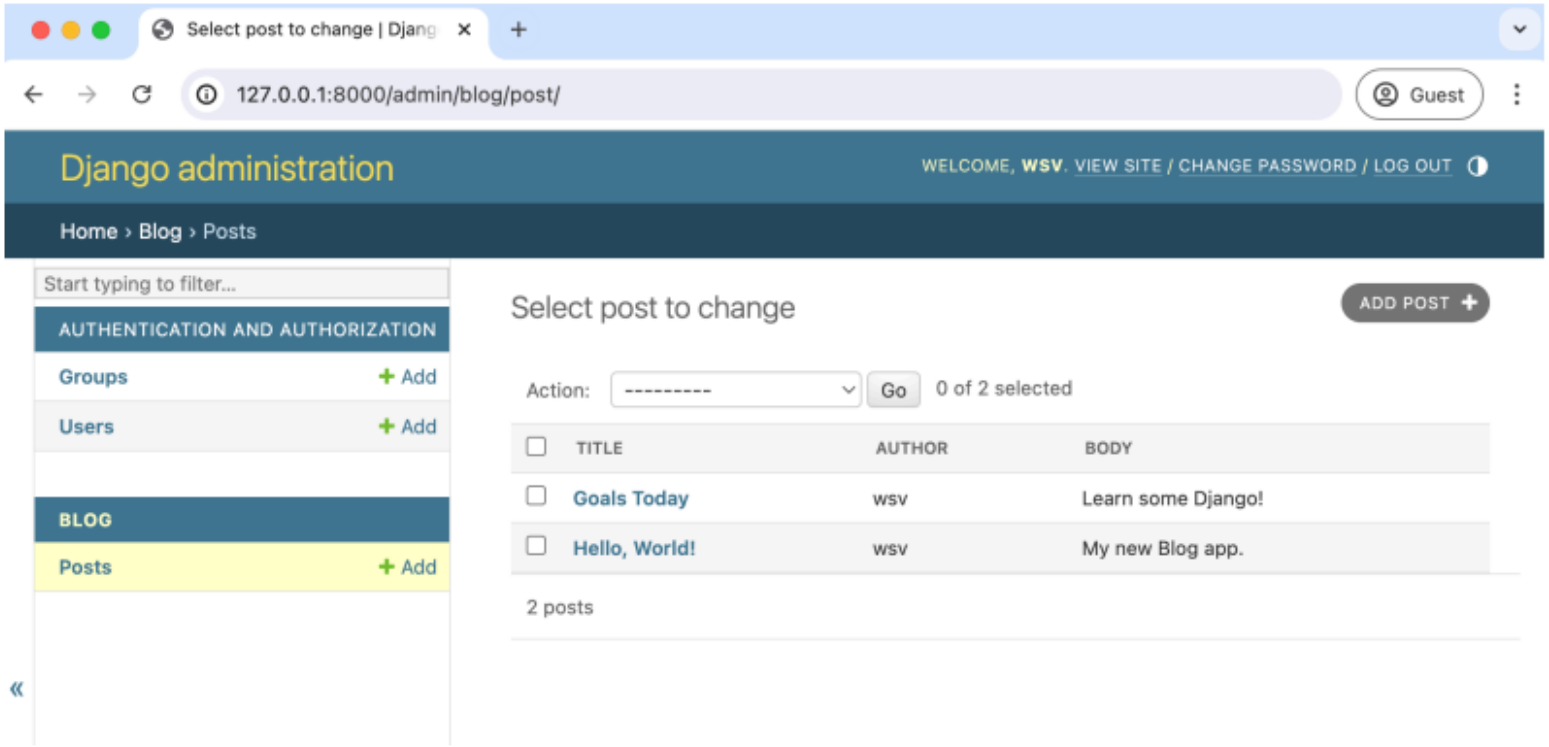
```
# blog/admin.py
from django.contrib import admin
from .models import Post


class PostAdmin(admin.ModelAdmin):  # new
    list_display = (
        "title",
        "author",
        "body",
    )


admin.site.register(Post, PostAdmin)  # new
```

To do this, we can extend ModelAdmin by creating a new class, PostAdmin. Within it, we can set list_display to control what is shown on the admin change list page. We also must register both the model, Post, and the extended ModelAdmin class we've created, PostAdmin, at the bottom of the file.

All three model fields will be visible if you refresh the admin change list page.



Admin Change List with All Fields

Static files

Static files are the Django community's term for **additional files commonly served on websites**, such as CSS, fonts, images, and JavaScript.

Even though we haven't added any to our project yet, we are already relying on core Django static files—custom CSS, fonts, images, and JavaScript—to power the look and feel of the Django admin.

In **production, things are more complex**, it is far more efficient to combine all static files across a Django project into **a single location** in production.

By default, Django will look within each app for a folder called “static”.

As Django projects grow in complexity over time and have multiple apps, it is often simpler to reason about static files if they are stored in a single, project-level directory. That is what we will do.

Code

```
# django_project/settings.py
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]  # new
```

Shell

```
(.venv) $ mkdir static/css
```

Add a new file within your text editor in this directory called `static/css/base.css`. What should we put in our file? How about changing the title to red?

Code

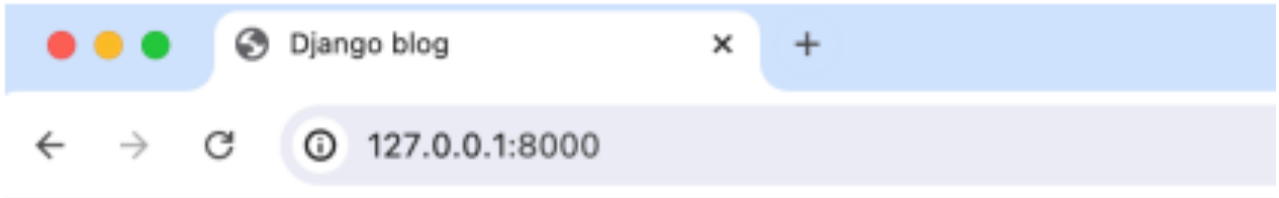
```
/* static/css/base.css */
header h1 a {
 color: red;
}
```

The last step is adding the static files to our templates by adding `{% load static %}` to the top of `base.html`. Because our other templates inherit from `base.html`, we only have to add this once. Include a new line at the bottom of the `<head></head>` code that explicitly references our new `base.css` file.

Code

```
<!-- templates/base.html -->
{% load static %}
<html>

<head>
 <title>Django blog</title>
 <link rel="stylesheet" href="{% static 'css/base.css' %}">
</head>
...
```



Django blog

Hello, World!

Author: wsv

My new Blog app.

Goals Today

Author: wsv

Learn some Django!

Then you will jazz up things by adding more css

Individual Blog Pages

Now, we can add functionality for individual blog pages. We need to create a new view, URL, and template to do that.

You should be getting a feel for the pattern.

Code

```
# blog/views.py
from django.shortcuts import render, get_object_or_404  # new
from .models import Post


def post_list(request):
    posts = Post.objects.all()
    return render(request, 'home.html', {'posts': posts})


def post_detail(request, pk):  # new
    post = get_object_or_404(Post, pk=pk)
    return render(request, "post_detail.html", {"post": post})
```

At the top of our views file, import the shortcut function **get_object_or_-404()**, which calls the get QuerySet method to return an object or raises a Http404 error if unsuccessful.

We will name the view function **post_detail** as it represents the detailed view of a blog post. It accepts two parameters: the first, request, is an instance of the **HttpRequest object**; the second, **pk**, is a parameter extracted from the URL that identifies the specific blog post to be displayed by its primary key.

Code

```
# blog/urls.py
from django.urls import path
from .views import post_list, post_detail  # new

urlpatterns = [
    path("post/<int:pk>/", post_detail, name="post_detail"),  # new
    path("", post_list, name="home"),
]
```

This pattern will look a bit strange initially as it is our first use of a Django path converter. Up to this point, we have hardcoded our URL routes, but it is **more common to use variables**.

In this case, we specify that **individual posts will start at post/,** but then we use **int** to specify that the **captured value from the URL** should be treated as an **integer** and the **variable's name passed to the view is pk**.

The last step is adding a new template file called `templates/post_detail.html` in your text editor. Then type in the following code:

Code

```
<!-- templates/post_detail.html -->
{% extends "base.html" %}

{% block content %}
<div class="post-entry">
 <h2>{{ post.title }}</h2>
 <p>{{ post.body }}</p>
</div>
{% endblock content %}
```

At the top, we specify that this template inherits from base.html. The template **context variable post** contains information from this particular blog post, and we can display individual fields using **post.title** and **post.body**.

Django blog

Hello, World!

My new Blog app.

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}
{% for post in posts %}
<div class="post-entry">
 <h2><a href="{% url 'post_detail' post.pk %}">{{ post.title }}</a></h2>
 <p>{{ post.body }}</p>
</div>
{% endfor %}
{% endblock content %}
```

We start by using Django's `url` template tag and specifying the URL pattern name of `post_detail`. If you look at `post_detail` in our URLs file, it expects to be passed an argument `pk` representing the primary key for the blog post. Fortunately, Django has already created and included this `pk` field on our `post` object, but we must pass it into the URL by adding it to the template as `post.pk`.

Currently, we are using the url template tag, which means that every time we want to display an individual blog post in this template or other templates, we must repeat the pattern {% url 'post_detail' post.pk %}. **If the URL pattern changes, we need to update every template where the URL is constructed**, which increases the risk of errors.

Code

```
# blog/models.py
from django.db import models
from django.urls import reverse  # new


class Post(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(
        "auth.User",
        on_delete=models.CASCADE,
    )
    body = models.TextField()

    def __str__(self):
        return self.title

    def get_absolute_url(self):  # new
        return reverse("post_detail", kwargs={"pk": self.pk})
```

At the top, we import reverse()[118], a utility function for our URLs. Then, we define `get_absolute_url` using `self` as the first parameter referring to the model instance on which the method is called. This is a standard practice in Python for instance methods. The `reverse()` function accepts the URL name and keyword arguments or "kwargs." In this case, we are setting the variable `pk` to equal the primary key of our model instance.

Code

```
<!-- templates/home.html -->
{% extends "base.html" %}

{% block content %}
{% for post in posts %}
<div class="post-entry">
 <h2><a href="{{ post.get_absolute_url }}">{{ post.title }}</a></h2> <!-- new -->
 <p>{{ post.body }}</p>
</div>
{% endfor %}
{% endblock content %}
```

URL paths can and do change over a project's lifetime. With the previous method, if we changed the post detail view and URL path, we'd have to go through all our HTML and templates to update the code, a very error-prone and hard-to-maintain process. By using get_absolute_url() instead, we have one place, the models.py file, where the canonical URL is set, so our templates don't have to change